THE OPERATOR SERIES

N° 01

The Claude Operator's Manual

Everything Anthropic's docs never explained in plain English. The setup, the tools, and the workflows that turn Claude from a chatbot into your operating system.

Elvin Peters

First Edition · 2026

The Claude Operator's Manual

Everything Anthropic's docs never explained in plain English.

Elvin Peters

First Edition · 2026 · Written for Claude as it stands today

BEFORE YOU BEGIN

How to Use This Manual

This book is built to be read in order the first time and raided for the rest of its life. The early chapters assume nothing. The later ones assume you have read the early ones. If you skip ahead to hooks or subagents and feel lost, the fix is almost always a chapter you jumped over.

The manual comes in four parts. **Part I** is hands-on with the Claude apps and needs no terminal. **Part II** explains how Claude actually works under the hood, the mental model everything else rests on. **Part III** is the operator's power tools in Claude Code, the command-line half where the real leverage lives. **Part IV** wires it all into repeatable workflows. The appendices at the back are pure reference: cheat sheets you will keep open in a second window.

Throughout, recurring boxes flag the things worth stopping for. A **Tip** is a shortcut worth adopting. A **Gotcha** is the thing that quietly bites people. A **Key Term** defines a word you will see again. **Do This** blocks are copy-exact steps, **Worked Examples** are full walk-throughs, and every chapter ends with a **Checklist** so you can confirm the capability actually landed. Code and configuration appear in labeled panels you can copy verbatim.

One promise and one caveat. The promise: read this cover to cover and you will operate Claude at a level most users never reach. The caveat: this product moves fast. Where a detail is the kind that changes between releases, this manual teaches the durable concept and points you to the official docs for the current specifics. It is dated for a reason. Treat the mental models as permanent and the exact flags as perishable.

THE SHIFT

The Operator's Mindset

Most people chat with Claude. They open a box, type a question, read the answer, and close the box. That works, and it is also using a workshop as if it were a vending machine.

An operator does something different. An operator sets Claude up: gives it memory so it stops forgetting, gives it rules so it stays on the rails, gives it tools so it can act, and gives it workflows so the same good result happens every time without babysitting. The difference between the two is not talent or a secret prompt. It is setup. Everything in this book is setup you can copy.

Hold one idea as you read. Claude is capable far past what a single chat box reveals, and the ceiling is set by how well you configure it, not by how cleverly you phrase one message. Learn the configuration, and the same model that gave you a decent paragraph will run a project. That is the whole game. Turn the page and let us set it up.

Contents

PART I · CLAUDE, HANDS-ON

1 The Claude Landscape 7
2 Talking to Claude Well 14
3 Projects, Knowledge & Artifacts 23
4 Skills & the Skill Creator 32
5 Connecting Claude to Your World 39

PART II · HOW CLAUDE WORKS

6 Inside the Machine 48
7 The API 54

PART III · CLAUDE CODE

8 Meet Claude Code 62
9 Memory & Files 71
10 Settings & Permissions 81
11 Plan Mode 93
12 Slash Commands & Master Commands 102
13 Skills in Depth 112
14 Hooks & Hard Rules 125
15 Subagents 137
16 MCP in Depth 147
17 Worktrees, Checkpoints & Sessions 158

PART IV · PUTTING IT TOGETHER

18 Operator Workflows 173
19 The Operator's Practice 184

APPENDICES

A The Day-One Setup Checklist 192
B Slash-Command Reference 194
C Hook-Event Reference 197
D settings.json Key Reference 201
E CLI Flag Reference 206
F Glossary 211
G Sources & Further Reading 214

Chapter Three

Projects, Knowledge & Artifacts

How to turn one-off chats into a reusable workspace that already knows your context, and into finished deliverables you can publish and share.

A plain chat forgets the moment you close it. Every new conversation starts cold, so you paste the same background, restate the same rules, and re-upload the same reference file you uploaded yesterday. For a quick question that is fine. For a body of work you return to every day, it is a tax you pay over and over, and it quietly caps how good the answers can get.

Two features exist to end that. A **Project** is a persistent workspace that remembers your instructions, your reference files, and every chat you hold inside it, so each new conversation walks in already briefed. An **Artifact** is a standalone thing Claude builds beside the chat: a document, a chart, a small web app, something you can edit, publish, and hand to someone else.

This chapter takes both past the one-line summary from Chapter 1. You will set up a Project the right way, learn what belongs in its standing instructions versus a single chat, load reference knowledge without choking the workspace, build and refine an Artifact through plain follow-ups, and finish with a clear rule for which container fits which job.

3.1 What a Project actually bundles

A Project ties three things together into one durable space. Understanding the three parts is most of the battle, because each one solves a different flavor of the "explain it again" problem.

KEY TERM

Project a persistent Claude workspace that combines custom instructions, an uploaded knowledge library, and all the chats you run inside it. Every new chat in the Project inherits the instructions and the knowledge automatically.

The first part is **custom instructions**: a standing brief that behaves like a system prompt scoped to this one workspace. It defines who Claude is being for you here, the tone to write in, and the rules to hold across every conversation. The second part is the **knowledge library**: files you upload once as reference, so Claude can consult them in any chat without you attaching them again. The third part is the **chats themselves**, all kept together in the Project so the work stays in one place instead of scattered across your history.

The payoff is that instructions and knowledge load into every new conversation for free. You open a fresh chat inside the Project and Claude already knows the brand voice, already has the style guide, already understands the standing rules. That is the whole reason Projects exist: a consistent context you set up once and reuse indefinitely.

▸ DO THIS · STAND UP YOUR FIRST PROJECT

1. **Open the Projects area.** Go to `claude.ai/projects`, or hover the left sidebar and click **Projects**. Choose to create a new one and give it a clear name, like the client, the course, or the research topic it will hold.
2. **Write the custom instructions.** Keep them general: the role Claude plays, the voice to use, and the constraints that are true in every chat. Section 3.2 covers what to put here and what to leave out.

3. **Add reference knowledge.** Upload the documents Claude should treat as source material: the brand guide, the spec, the transcript set. These become searchable in every chat without re-attaching.
4. **Start a chat inside the Project.** Ask something that leans on both the instructions and the files. When the answer reflects your voice and your documents without you pasting either, the workspace is working.

Projects are available on every plan, including Free. Free accounts can create a handful of them; paid plans (Pro, Max, Team, and Enterprise) remove that ceiling and let you make as many as you want. If you are running several distinct bodies of work, that alone is a reason to be on a paid tier.

> *I* TIP
>
> If your account has memory turned on, each Project keeps its own **separate** memory. What Claude learns inside a client Project stays inside that Project and never surfaces in an unrelated personal one. That isolation is a confidentiality boundary you can rely on, so keep genuinely separate work in genuinely separate Projects rather than one catch-all space.

3.2 Instructions that belong in the Project, and the ones that do not

The most common Project mistake is stuffing the custom instructions with today's task. The instructions load into every conversation, so anything you write there applies to all of them. That makes the box perfect for durable context and wrong for one-off asks.

Put the **standing** things in the Project instructions. The role Claude is playing (a copy editor, a financial analyst, a patient tutor). The voice and formatting you always want. The hard constraints that never change, like a reading level to hold to, a term to avoid, or a currency to price in. Official guidance is to keep this section concise and focused on general context, because a short, sharp brief is followed more reliably than a long rambling one.

Put the **specific** things in the individual chat. The document you want edited today. The exact question. The one-time exception to a usual rule. Task detail lives in the conversation where it applies, so it does not silently bleed into next week's unrelated work in the same Project.

QUICK REFERENCE · PROJECT INSTRUCTIONS VS THE CHAT

Put in Project instructions	Put in the individual chat
The role Claude plays for this workspace	The specific task you need done today
Tone and formatting you always want	A one-time deviation from the usual tone
Standing rules that hold every time	The exact document or data for this request
Vocabulary, audience, reading level	Follow-up questions and revisions

There is a second lever worth knowing, because people reach for the Project instructions when they actually want it. A **Style** controls how Claude writes, the tone and length and formatting, and you can apply it to any chat regardless of Project. Presets like Concise or Explanatory ship built in, and you can create your own from a writing sample or a description. When you want the same voice everywhere you go, a Style travels with you across Projects. When you want facts, rules, and role fixed for one workspace, that is the Project instructions. They compose cleanly, so a formal Style can run inside a client Project without conflict.

3.3 Loading knowledge without choking the workspace

Project knowledge is reference material Claude can pull from in any chat. You upload the files once and they stay put. Common formats work directly, including PDF, DOCX, CSV, TXT, HTML, RTF, ODT, and EPUB, which covers most of what a working document set is made of.

The knowledge library has room, and it also has limits, so treat it like a shelf rather than a junk drawer. There is a per-file size cap and an overall budget for how much total material a Project can hold in active context, roughly the equivalent of a few hundred pages of English text before Claude has to start

being selective about what it pulls in. The exact figures shift as the product evolves, so check the official Projects documentation for the current per-file size and total capacity rather than trusting a number you memorized months ago.

> **UNDER THE HOOD**
>
> You do not have to keep your library tiny to keep it fast. On paid plans, as a Project's knowledge grows past what fits comfortably in context, Claude switches to retrieval mode: it searches the library and pulls only the chunks relevant to your current question instead of loading every file every turn. That is why a large, well-organized knowledge base still answers quickly. The practical effect is a much larger effective capacity than the raw context window suggests, and it happens automatically with no toggle to flip.

Two habits keep a knowledge base useful. First, upload the durable reference material, the things Claude should treat as ground truth every time, and leave the disposable, one-off files for a normal chat attachment instead. Second, give files clear names and prune what has gone stale, because a library full of outdated drafts pollutes answers just as surely as no library at all.

> **GOTCHA**
>
> A scanned or image-heavy PDF loses its visual detail past a certain page count. Claude reads the full text of a long document, but charts, diagrams, and handwriting in a very long scan may not be analyzed the way they are in a short one. The upload still succeeds, so the degrading is silent. If a chart deep inside a 400-page scan matters, pull that page out into its own short file rather than trusting the big upload to see it.

3.4 Artifacts: building something you can keep

A Project organizes your inputs. An Artifact is an output. It opens in a side panel next to the conversation and holds a single standalone deliverable that lives on its own, separate from the chat scrolling past it.

KEY TERM

Artifact a standalone piece of work Claude creates in a side panel: a document, a chart or diagram, an SVG image, a single-page web app, a small tool. You refine it with plain-language follow-ups and can publish it to a shareable link.

You get an Artifact in one of three ways. Describe what you want in a normal chat and Claude opens one when the output calls for it. Ask outright, with something like "make this an artifact." Or start from the Artifacts area, describe the goal, answer a clarifying question or two, and let Claude produce a working first version in a few turns. All three land in the same place: a live panel you can watch take shape.

Plenty of things make good Artifacts. A polished document such as a report, memo, or proposal. A chart or a flowchart. An SVG graphic. A single-page website in HTML, CSS, and JavaScript, static or interactive. A small calculator or tool your team can actually use. The common thread is a self-contained deliverable, something with edges you would want to hand to another person, rather than a passing answer inside a conversation.

The refining loop is where Artifacts earn their keep. Once a first version exists, you steer it in plain language. "Make the buttons bigger." "Add a total row." "Use a warmer color for the header." Each follow-up edits the Artifact in place, so you shape it by describing changes the way you would to a designer sitting next to you. When something breaks, a **Fix with Claude** button takes a pass at the problem, or you can just describe what went wrong and let Claude repair it.

▸ WORKED EXAMPLE · FROM ROUGH ASK TO SHARED TOOL

You want a simple tip calculator for your restaurant staff. You type: "Build a tip calculator where I enter the bill and pick a tip percentage, and it shows the total and the per-person split." Claude opens an Artifact with a working single-page app in a few turns.

You try it and refine by talking. "Add an 18, 20, and 22 percent button." "Let me set the number of people with a plus and minus." "Make the total bigger

and easier to read on a phone." Each request updates the same Artifact. When a button misbehaves, you click Fix with Claude and it sorts itself out.

Satisfied, you click **Publish**. Claude gives you a link. You send it to the team, and they open the calculator in a browser and use it, with no Claude account of their own required. A rough sentence became a real tool your staff uses, and you never wrote a line of code.

Publishing is the last step. A **Publish** button at the bottom of the Artifact panel turns it into a shareable link. Most published Artifacts are view-only for anyone with the link, and viewers do not need a Claude account to open them. Other Claude users can **Remix** a published Artifact, taking their own editable copy to build from without touching your original.

> **TIP**
>
> If your first attempt is close and you want to try a bolder direction without losing it, fork the conversation. Forking branches the thread so you can explore a different version while the original Artifact stays intact. It is the safe way to gamble on a big change when you already have something decent in hand.

> **GOTCHA**
>
> An Artifact that itself calls Claude, the kind that acts like a small assistant with built-in knowledge, behaves differently when shared. A viewer has to sign in to use it, because it needs to talk to the model on their behalf. A plain static Artifact opens for anyone with the link. And Artifact availability on mobile has moved around over time, so if you rely on building them from your phone, confirm current mobile support in the app before you count on it.

3.5 What to use when

With chat, Projects, and Artifacts all in hand, the only real skill left is choosing the right container before you start. The choice comes down to two questions. Are you returning to this context repeatedly, and are you producing a standalone thing you will keep?

Reach for a **Project** when the answer to the first question is yes: you have a body of knowledge and a consistent set of rules you will lean on again and again. A client and all their brand assets. A research corpus you keep querying. A course you are building over weeks. The setup cost pays for itself the second time you open a chat and Claude already knows everything. Reach for a plain chat when it is a one-off with no reusable context, where the setup would cost more than the task.

Reach for an **Artifact** when the answer to the second question is yes: the goal is a deliverable with edges, something to edit, publish, and hand off. A document, a chart, a tool, a page. The two features stack, so the strongest setup is often both at once. You build inside a Project so Claude has your context, and the chat produces an Artifact you publish and share.

QUICK REFERENCE · WHICH CONTAINER FOR THE JOB

If the job is	Use	Why
A quick question with no reusable context	**Plain chat**	Zero setup, fastest path to an answer
Repeated work against the same knowledge and rules	**Project**	Instructions and files load into every chat automatically
Producing a document, chart, app, or tool to keep	**Artifact**	A standalone thing you can edit, publish, and share
A reusable deliverable that needs your standing context	**Artifact inside a Project**	Your knowledge and voice, producing a shareable output
The same voice applied everywhere you write	**Style**	Travels with you across chats and Projects

One reflex captures all of it. Before you type, ask whether you will come back to this context and whether you want to keep what comes out. A yes to the first points at a Project. A yes to the second points at an Artifact. A no to both means a plain chat is exactly right, and there is no shame in that being most of your day.

CHAPTER 3 · CHECKLIST

You can now

- Name the three things a Project bundles: custom instructions, a knowledge library, and its chats.
- Stand up a Project and confirm its instructions and files load into every new chat automatically.
- Split standing context into Project instructions and keep one-off asks in the individual chat.
- Upload reference knowledge in the right formats and keep the library lean, named, and pruned.
- Create an Artifact, refine it with plain-language follow-ups, and repair it with Fix with Claude.
- Publish an Artifact to a shareable link and know when a viewer needs an account to use it.
- Choose the right container for a job, and combine a Project and an Artifact when both fit.